Lecture 3: Policies, Timing, Populations

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 13, 2026

University of Toronto

Policy Function

Policy Function: $c^\star$

The policy function is the **argmax** version of the **max** Bellman operator:

$$\mathcal{T}v(b) = \max_{c \in [0,b]} u(c) + \beta v\big(R(b-c)+z\big) \qquad \text{(Bellman Operator)}$$

$$c^\star(b) = \arg\max_{c \in [0,b]} u(c) + \beta v\big(R(b-c)+z\big) \qquad \text{(Policy Function)}$$

- Once the true V is solved, $\arg\max$ gives you the optimal policy $c^\star$.
- More deeply, any v gives you some policy function.
  - v can be thought of as representing beliefs about the value of being in various states.

Marginal Propensity to Consume (MPC)

The slope $\partial c^\star / \partial b$ is the **Marginal Propensity to Consume (MPC)**

Generally (theoretically and empirically) higher for poorer households, low for rich.

Can be thought of as a function of how **constrained** households are.

Conventions

Conventions

- Now that we have some intuition, let's set up some conventions that will pay off later.
- Think of the state variable z as **human capital**,following a **Markov process** Π_z
- Period income is zw, where w is the wage level. For now, we'll normalize $w = 1$.
- The **state** is now (b, z).
- On computer, each V is now a **matrix**. Rows are wealth b, columns are human capital z.

Stages

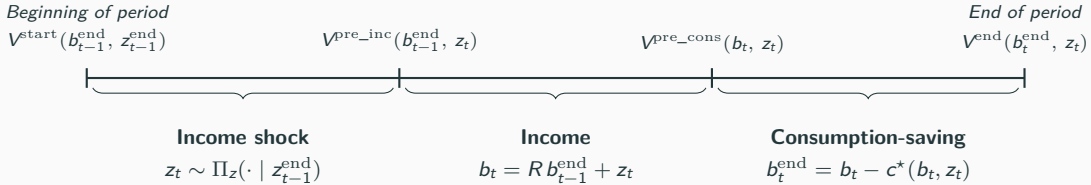
Each **model feature** can be thought of as a **Stage**:

1. Income Shock
2. Income
3. Consumption-Saving

Each **Stage** is a **simple sub-period**.

- More on this later, but **functional composability** of stages is analogous to the **additive composability** of continuous-time household models.
- This can allow us to build **complex** models by composing **Stages**.

Within-Period Timeline



Each **Stage** has different **state variables** and a different **intermediate value function**.

We can iterate V^{end} backward one stage at a time.

Stages

Stage 1 — Income Shock

- $(\Pi_z)_{ij} = \Pr(z_t = j \mid z_{t-1}^{\text{end}} = i)$, row-stochastic.
- **Backward** (value functions).

$$V^{\text{start}} = V^{\text{pre\_inc}} \Pi_z^\top.$$

Each $(b^{\text{end}}, z_{t-1}^{\text{end}})$ entry of V^{start} is an expectation over z_t given z_{t-1}^{end}.

(Note: To see why matrix multiplication can represent backwards-iteration of V through a Markov chain, see standard references on Markov chains.)

Stage 2 — Income

This stage just represents the **receipt** of income,

$$b_t = R\, b_{t-1}^{\text{end}} + z_t,$$

effectively a change-of-variables.

Explicitly, the backwards step is,

$$V^{\text{pre\_inc}}(b_{t-1}^{\text{end}}, z) \;=\; V^{\text{pre\_cons}}\big(R\, b_{t-1}^{\text{end}} + z,\; z\big).$$

In the example code, we do this by **reinterpolating** $V^{\text{pre\_cons}}$ at the new points $R\, b_{t-1}^{\text{end}} + z$.

Stage 3 — Consumption-Saving

This stage represents the argmax step,

$$b_t^{\text{end}} \;=\; b_t - c^\star(b_t, z_t).$$

Explicitly, the backwards step is,

$$V^{\text{pre\_cons}}(b, z) \;=\; \max_{b^{\text{end}} \in b_{\text{grid}}} u(b - b^{\text{end}}) + \beta V^{\text{end}}(b^{\text{end}}, z).$$

Here, we fold in the time discounting by β in the consumption-saving stage, but in general it can be thought of as really a separate stage.

Backwards Iteration

```
"Iterate V_end backwards by one full period."
function bellman_operator(V_end, params)β
    (;) = params

    # Stage 3
    V_post_inc = consumption_saving_backward(V_end, params)

    # Stage 2
    V_post_inc_shock = income_backward(V_post_inc, params)

    # Stage 1
    V_start = income_shock_backward(V_post_inc_shock, params)

    # Stage 0
    V_end_new = β .* V_start

    return V_end_new
end
```

Backwards iteration of V is just **functional composition** of the backwards-iteration functions of each stage

Simulating a Population

Heterogeneity for (Almost) Free

- We have been thinking of V and c^* as the value and policy of a single household at different **possible** states.
- But we can also think of V and c^* as the value and policy of **contemporaneous** households in a **population** with different idiosyncratic states.
- Since we have already computed V on the whole grid, this lets us add heterogeneity very cheaply.
- We just need to think about how to **represent** and **simulate** a population of households.

A Population is a Vector (or Matrix)

We can represent the *population* as a discrete distribution Λ over the wealth grid:

$$\Lambda_i = \text{share of households at } b_i.$$

That is, just like V, it is a vector (or a matrix) over the discretized state space.

(Note: Matrices are vectors.)

Aggregate Moments

- Aggregate welfare is then an inner product: $\bar{V} = \sum_i V(b_i)\,\Lambda_i$. Same for $\bar{C}$, $\bar{K}$, etc.
- Indeed, **any** moment of the distribution (e.g. aggregate capital, consumption, etc.) can be thought of as an inner product against Λ.

Forward and Backward, All Three Stages

	Backward (value)	**Forward** (distribution)
Stage 1	$V^{\text{start}} = V^{\text{pre\_inc}} \, \Pi_z^\top$	$\Lambda^{\text{post}} = \Lambda^{\text{pre}} \, \Pi_z$
Stage 2	$V^{\text{pre\_inc}}(b^{\text{end}}, z) = V^{\text{pre\_cons}}(R\, b^{\text{end}} + z,\, z)$	$(b^{\text{end}}, z) \to (R\, b^{\text{end}} + z,\, z)$
Stage 3	$V^{\text{pre\_cons}}(b, z) = \max_{b^{\text{end}}} u(b - b^{\text{end}}) + V^{\text{end}}(b^{\text{end}}, z)$	$(b, z) \to \big(b - c^\star(b, z),\, z\big)$
Dynamic Boundary Condition	$V^{\text{end}_t} = \beta \, V^{\text{start}}_{t+1}$	$\Lambda^{\text{start}}_t = \Lambda^{\text{end}}_{t+1}$
Steady State	$V^{\text{end}} = \beta \, V^{\text{start}}$	$\Lambda^{\text{start}} = \Lambda^{\text{end}}$

Backward: $V^{\text{end}} \to V^{\text{pre\_cons}} \to V^{\text{pre\_inc}} \to V^{\text{start}}$.

Forward: $\Lambda^{\text{start}} \to \Lambda^{\text{pre\_inc}} \to \Lambda^{\text{pre\_cons}} \to \Lambda^{\text{end}}$.

Stage 1: Markov Income Shock

Forward:

```
# Λ is an N_b × N_z matrix: rows index wealth, columns index z state.
# Markov along the z dimension is a single matrix multiplication.
income_shock_forwardΛ(, Π_z) = Λ * Π_z
```

- The mass at (b_i, z_j) flows to $(b_i, z_{j'})$ with weight $(\Pi_z)_{jj'}$.
- Markov along one dimension is just a matrix multiplication.

simulate_population_forward

```
"Simulate population forward through all three Stages: Λ→  income shock → income → consumption saving."
function simulate_population_forwardΛ(, b_inds_new, params)
    # Stage 1Λ
     = income_shock_forwardΛ(, paramsΠ._z)

    # Stage 2Λ
     = income_forwardΛ(, params)

    # Stage 3Λ
     = consumption_saving_forwardΛ(, b_inds_new, params)

    return Λ
end
```

Forward simulation is just **functional composition** of the forward-simulation functions of each stage.

Solving for Steady State

1. Guess v and Λ.
2. Iterate v backwards until it converges to V.
3. Simulate Λ forward until it converges to its steady state.

(Note: The conditions under which Λ converges are subtler than for V.)